CardForge Studio - Agent Execution Spec

Guía de Arquitectura de Código y Estructura de Módulos para Automatización de Agentes IA

1. Estructura de Directorios del Proyecto (Clean Architecture)

El agente debe organizar el proyecto Flutter Desktop utilizando una separación estricta de responsabilidades en capas:

Directorio	Propósito y Responsabilidad de Clases
`lib/core/`	Utilidades globales, manejo de archivos locales y configuración de temas.
`lib/features/editor/`	Lienzo visual, interactividad de capas, transformaciones y motor de plantillas.
`lib/features/database/`	Gestión de registros tabulares (CSV/JSON), enlace de variables y tablas de datos.
`lib/features/export/`	Motor de renderizado off-screen a 300 DPI y exportación masiva a PDF/PNG.
`lib/features/playtest/`	Módulo interactivo de simulación de mesa y pruebas de balance de mazos.

Dependencias Clave de Pubspec.yaml

- `flutter_riverpod` o `flutter_bloc` : Gestión de estado reactiva centralizada para el mazo y las variables.
- `pdf` & `printing` : Generación de documentos listos para impresión profesional.
- `archive` & `path_provider` : Empaquetado y gestión del formato de archivo local de proyectos (.cfproj).
- `csv` & `hive` : Parsing rápido de fuentes de datos tabulares y persistencia ligera local.

2. Modelos de Datos Esenciales (Dart Entities)

Modelo	Atributos Clave a Implementar
`CardProject`	`id` , `name` , `widthMm` , `heightMm` , `List<CardTemplate> templates` , `List<Map<String, dynamic>> rows`
`CardTemplate`	`id` , `name` , `List<LayerItem> layers`
`LayerItem`	`id` , `type` (text, image, shape), `x` , `y` , `width` , `height` , `String bindingKey` , `Map conditionalRules`

3. Algoritmos Críticos para el Agente (Instrucciones de Implementación)

El agente debe priorizar la construcción de los siguientes subsistemas lógicos críticos para el funcionamiento central de la aplicación:

A. Motor de Sustitución de Variables y Reglas Condicionales

- El renderizador de texto debe interpretar patrones tipo `{{variable_name}}` buscando la clave correspondiente en la fila de datos activa.
- Implementar un evaluador de reglas booleanas simples para capas condicionales (ej. `if: cost > 5` evalúa la expresión contra el mapa de datos y oculta/muestra la capa o altera su color de borde).

B. Motor de Renderizado Off-Screen (Exportación Masiva 300 DPI)

- Utilizar un widget `RepaintBoundary` combinado con `RenderRepaintBoundary.toImage(pixelRatio: 3.5)` para transformar dinámicamente cada carta generada en un búfer de imagen en alta resolución.
- Iterar sobre todas las filas de la base de datos, inyectar los datos en la plantilla base en segundo plano (usando `compute` o streams) y exportar como lote empaquetado en ZIP o PDF multipágina.

C. Lienzo Interactivo con Snapping (Editor Visual)

- Construir un widget de lienzo basado en gestos y transformaciones matrices donde el usuario pueda mover elementos con precisión de milímetros.
- Implementar líneas guia automáticas (snapping) calculando distancias mínimas entre los bordes de las capas vecinas al arrastrar con el mouse.

4. Secuencia de Desarrollo Recomendada para el Agente

Fase	Hito Técnico	Objetivo Comprobable
Fase 1	Estructura Base & Estado	Creación de modelos de datos, inyección de dependencias y UI principal con multi-panel.
Fase 2	Editor Visual de Capas	Canvas interactivo con posicionamiento, selección de capas y propiedades en inspector.
Fase 3	Enlace de Datos (Data-Binding)	Importador de CSV/JSON conectado reactivamente a los campos de texto de la carta.
Fase 4	Exportador & Playtest	Generador de imágenes en lote de alta resolución y simulación de mesa virtual.